

Practical-1

Aim: To understand the overall programming architecture using Map Reduce API.

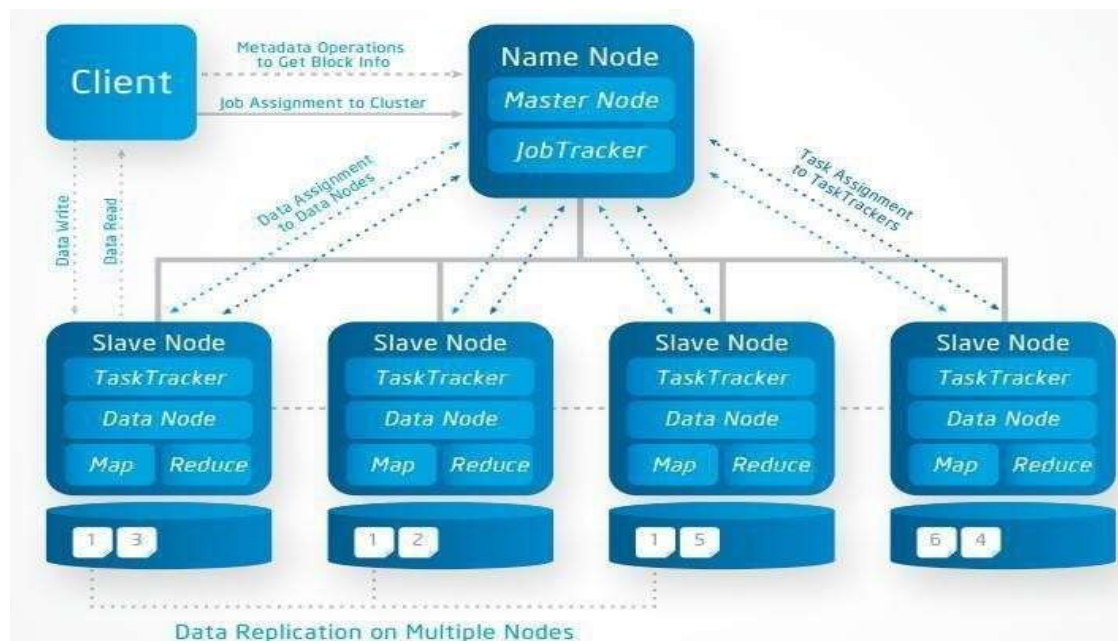
Description:

HDFS supports the rapid transfer of data between compute nodes. At its outset, it was closely coupled with MapReduce, a programmatic framework for data processing.

When HDFS takes in data, it breaks the information down into separate blocks and distributes them to different nodes in a cluster, thus enabling highly efficient parallel processing.

Moreover, the Hadoop Distributed File System is specially designed to be highly fault-tolerant. The file system replicates, or copies, each piece of data multiple times and distributes the copies to individual nodes, placing at least one copy on a different server rack than the others.

As a result, the data on nodes that crash can be found elsewhere within a cluster. This ensures that processing can continue while data is recovered.



Name Node, a master server that manages the file system namespace and regulates access to files by clients. The Name Node executes file system namespace operations like opening, closing, and renaming files and directories. Determines mapping of blocks to Data nodes

The Job Tracker is the service with in Hadoop that farms out MapReduce tasks to specific nodes in the cluster. Client applications submit jobs to the Job tracker. The Job Tracker submits the work to the chosen Task Tracker nodes.

Secondary Name Node in Hadoop is a specially dedicated node in HDFS cluster. Whose main function is to take checkpoints of the file system metadata present on name node. It is not a backup name node. It just checkpoints name node's file system namespace.

A Data Node stores data in the [Hadoop Distributed File System]. This functional file system has more than one Data Node, with data replicated across them. On start-up, a Data Node connects to the Name Node; spinning until that service comes up. It then responds to requests from the Name Node for file system.

A Task Tracker is a node in the cluster that accepts tasks - Map, Reduce process from a Job Tracker. Every Task Tracker is configured with a set of slots, these indicate the number of tasks that it can accept.

Step-1: SOFTWARES REQUIRED

Hadoop 1.2.0-bin.tar.gz

Jdk 7u67-linux-i586.tar.gz

Link for JDK <http://www.oracle.com/technetwork/java/javase/downloads/index.html>

Link of Hadoop : <http://mirror.fibergrid.in/apache/hadoop/common/hadoop-1.2.0/>

STEP-2 Untar the software

Open the terminal window

Type the command : cd Desktop

Type ls command

```
tar -zxvf jdk-7u67-linux-i586.tar.gz
```

```
tar -zxvf hadoop-1.2.0-bin.tar.gz
```

Step 3 Bashrc configurations

Open the terminal type the command

```
sudo gedit ~/.bashrc
```

```
export JAVA_HOME=/home/user/Desktop/jdk1.7.0_67
```

```
export PATH=$PATH:$JAVA_HOME/bin
```

```
export HADOOP_HOME=/home/user/Downloads/hadoop-1.2.0
```

```
export PATH=$PATH:$HADOOP_HOME/bin
```

Open new terminal and check for java version and Hadoop version.

Step-4 Hadoop Configuration Files

1. Core-site.xml

(Configuration setting for Hadoop core, such as I/O setting that are common to HDFS and MapReduce)

2. Hdfs-Site.xml

(Configuration setting for HDFS daemons the name node, Secondary name node and data node and Replication factor as well)

3. Mapred-site.xml

(configuration setting for MapReduce daemons: the job tracker and the task tracker)

4. Hadoop-Env.sh

(Environment variables that are used in the scripts to run Hadoop)

Open the new terminal window

Go to the Hadoop folder

```
cd Desktop
```

```
cd Hadoop (Press tab) for path
```

```
cd conf
```

```
Type ls
```

All the above mentioned files are listed

Go to terminal and type

sudo gedit core-site.xml

```
<configuration>
<property>
<name>fs.default.name</name>
<value>hdfs://localhost:9000</value>
</property>
</configuration>
```

Go to terminal and type

sudo gedit hdfs-site.xml

```
<configuration>
<property>
<name>dfs.replication</name>
<value>1</value>
<name>dfs.name.dir</name>
<value>/home/user/Downloads/hadoop-1.2.0/name/data</value>
</property>
</configuration>
```

Go to terminal and type

sudo gedit mapred-site.xml

```
<configuration>
<property>
<name>mapred.job.tracker</name>
<value>hdfs://localhost:9001</value>
</property>
</configuration>
```

Go to terminal and type

sudo gedit hadoop-env.sh

```
export JAVA_HOME=/home/user/Desktop/jdk1.7.0_67
```

STEP-5 SSh Configuration

```
sudo apt-get install ssh
```

```
ssh-keygen -t rsa -P ""
```

Sharing the public key with host

```
ssh-copy-id -i ~/.ssh/id_rsa.pub user@ubuntuvms and check
ssh localhost (It should not ask password)
```

STEP 6 Formatting the Name Node

```
$ hadoop namenode -format
```

format a new distributed file system

Process creates an empty file system for creating the storage directories

STEP-7&8 Starting Hadoop Daemons

Open the terminal window and type

```
$ start-all.sh
```

and type

```
$ jps
```

```
user@ubuntuvirtual-machine:~$ start-all.sh
```

```
Warning: $HADOOP_HOME is deprecated.
```

```
starting namenode, logging to /home/user/Downloads/hadoop-1.2.0/libexec/../logs/hadoop-user-namenode-ubuntuvirtual-machine.out
```

```
localhost: datanode running as process 2978.
```

```
localhost: secondarynamenode running as process 3123.
```

```
jobtracker running as process 3204.
```

```
localhost: tasktracker running as process 3342.
```

```
user@ubuntuvirtual-machine:~$ jps
```

```
4020 Jps
```

```
3342 TaskTracker
```

```
3204 JobTracker
```

```
3123 SecondaryNameNode
```

```
3606 NameNode
```

```
2978 DataNode
```

MapReduce and HDFS are the two major components of Hadoop which makes it so powerful and efficient to use.

MapReduce is a programming model used for efficient processing in parallel over large data-sets in a distributed manner.

The data is first split and then combined to produce the final result.

The libraries for MapReduce is written in so many programming languages with various different-different optimizations.

The purpose of MapReduce in Hadoop is to Map each of the jobs and then it will reduce it to equivalent tasks for providing less overhead over the cluster network and to reduce the processing power.

The MapReduce task is mainly divided into two phases Map Phase and Reduce Phase.

- 1.map(), filter(), and reduce() in Python.
2. These functions are most commonly used with Lambda function.

1.map():

"A map function executes certain instructions or functionality provided to it on every item of an iterable."The iterable could be a list, tuple, set, etc.

SYNTAX:

map(function, iterable)

Example:

```
items = [1, 2, 3, 4, 5]
```

```
a=list(map((lambda x: x **3), items))
```

```
print(a)
```

#Output:

```
items = [1, 2, 3, 4, 5]
a=list(map((lambda x: x **3), items))
print(a)
```

```
[1, 8, 27, 64, 125]
```

The map()function passes each element in the list to a lambda function and returns the mapped object.

2. filter():-

"A filter function in Python tests a specific user-defined condition for a function and returns an iterable for the elements and values that satisfy the condition or, in other words, return true."

SYNTAX:

filter(function, iterable)

Example:

```
a = [1,2,3,4,5,6]
```

```
b = [2,5,0,7,3]
```

```
c= list(filter(lambda x: x in a, b))
```

```
print(c)
```

#Output

```
a = [1,2,3,4,5,6]
b = [2,5,0,7,3]
c= list(filter(lambda x: x in a, b))
print(c)
```

```
[2, 5, 3]
```

3.reduce():

"Reduce functions apply a function to every item of an iterable and gives back a single value as a resultant".

we have to import the reduce function from functools module using the statement

SYNTAX:

reduce(function, iterable)

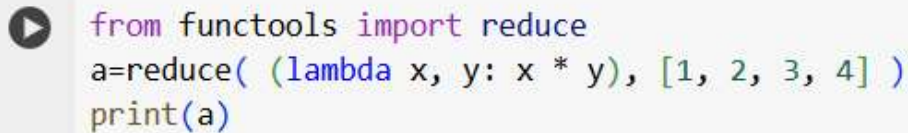
Example:

from functools import reduce

a=reduce((lambda x, y: x * y), [1, 2, 3, 4])

print(a)

#Output:

A screenshot of a code editor showing Python code. The code is:

```
from functools import reduce
a=reduce( (lambda x, y: x * y), [1, 2, 3, 4] )
print(a)
```

 To the left of the code is a play button icon. Below the code is a swap icon and the number 24.

⇄ 24

Practical-2

Aim: Write a program of Word Count in Map Reduce over HDFS.

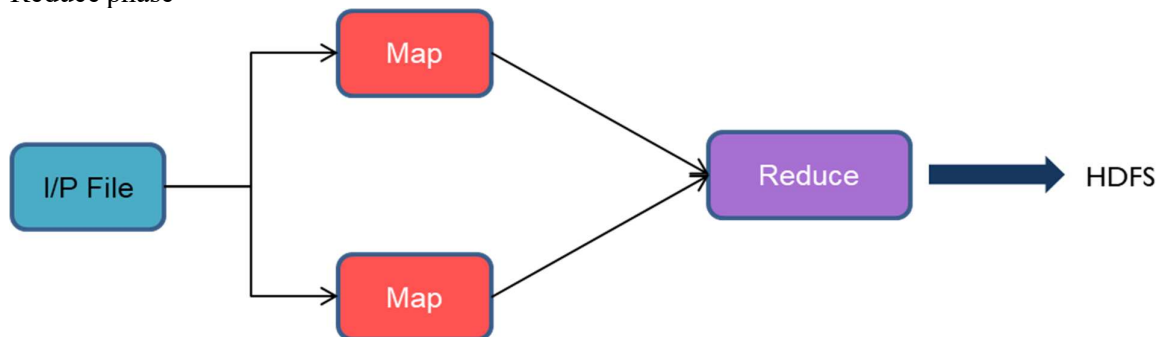
Description:

MapReduce is a framework for processing large datasets using a large number of computers (nodes), collectively referred to as a cluster. Processing can occur on data stored in a file system (HDFS). A method for distributing computation across multiple nodes. Each node processes the data that is stored at that node.

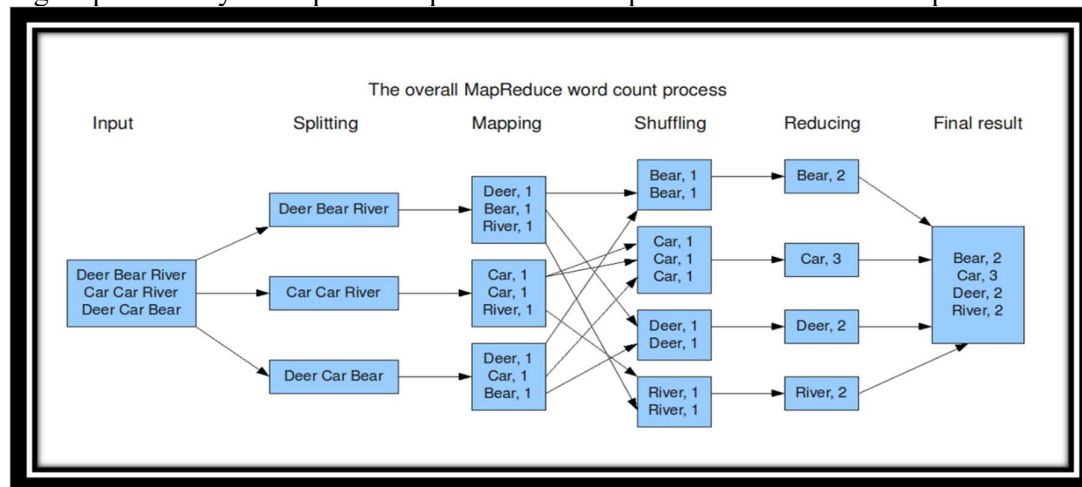
Consists of two main phases

Mapper Phase

Reduce phase



Input data set is split into independent blocks – processed in parallel. Each input split is converted in Key Value pairs. Mapper logic processes each key value pair and produces an intermediate key value pairs based on the implementation logic. Resultant key value pairs can be of different type from that of input key value pairs. The output of Mapper is passed to the reducer. Output of Mapper function is the input for Reducer. Reducer sorts the intermediate key value pairs. Applies reducer logic upon the key value pairs and produces the output in desired format. Output is stored in HDFS



Execution Step:

<http://content.udacity-data.com/courses/ud617/Cloudera-Udacity-Training-VM-4.1.1.c.zip>

Create the jar file of this program and name it wordcount.jar.

Run the jar file

hadoop fs -mkdir /input

hadoop fs -put /home/training/Desktop/sample.txt /input

hadoop jar /home/training/Desktop/wc.jar wordcount /input/sample.txt /output

Output:

```
hadoop fs -cat /output/part-00000
```

Word Count Java Program :

```
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.FileInputFormat;
import org.apache.hadoop.mapred.FileOutputFormat;
import org.apache.hadoop.mapred.JobClient;
import org.apache.hadoop.mapred.JobConf;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

public class wordcount extends Configured implements Tool {

    @Override
    public int run(String[] args) throws Exception {

        if(args.length<2)
        {
            System.out.println("Plz Give Input Output Directory Correctly");
            return -1;
        }

        JobConf conf = new JobConf(wordcount.class);
        FileInputFormat.setInputPaths(conf,new Path(args[0]));
        FileOutputFormat.setOutputPath(conf, new Path(args[1]));
        conf.setMapperClass(wordmapper.class);
        conf.setReducerClass(wordreducer.class);
        conf.setMapOutputKeyClass(Text.class);
        conf.setMapOutputValueClass(IntWritable.class);
        conf.setOutputKeyClass(Text.class);
        conf.setOutputValueClass(IntWritable.class);
        JobClient.runJob(conf);
        return 0;
    }

    public static void main(String args[]) throws Exception
    {
        int exitcode = ToolRunner.run(new wordcount(), args);
        System.exit(exitcode);
    }
}
```

```
import java.io.IOException;
```

```
import org.apache.hadoop.io.IntWritable;  
import org.apache.hadoop.io.LongWritable;  
import org.apache.hadoop.io.Text;  
import org.apache.hadoop.mapred.MapReduceBase;  
import org.apache.hadoop.mapred.Mapper;  
import org.apache.hadoop.mapred.OutputCollector;  
import org.apache.hadoop.mapred.Reporter;
```

```
public class wordmapper extends MapReduceBase implements  
Mapper<LongWritable,Text,Text,IntWritable>  
{  
    public void map(LongWritable key, Text value,  
                    OutputCollector<Text, IntWritable> output, Reporter r)  
        throws IOException {  
        String s =value.toString();  
        for(String word:s.split(" "))  
        {  
            if(word.length()>0)  
            {  
                output.collect(new Text(word), new IntWritable(1));  
            }  
        }  
    }  
}
```

```
import java.io.IOException;  
import java.util.Iterator;
```

```
import org.apache.hadoop.io.IntWritable;  
import org.apache.hadoop.io.Text;  
import org.apache.hadoop.mapred.MapReduceBase;  
import org.apache.hadoop.mapred.OutputCollector;  
import org.apache.hadoop.mapred.Reducer;  
import org.apache.hadoop.mapred.Reporter;
```

```
public class wordreducer extends MapReduceBase implements  
Reducer<Text,IntWritable,Text,IntWritable>  
{  
  
    public void reduce(Text key, Iterator<IntWritable> values,
```

OutputCollector<Text, IntWritable> output, Reporter r)
throws IOException {

```
int count=0;
while(values.hasNext())
{
    IntWritable i= values.next();
    count+= i.get();
}
output.collect(key, new IntWritable(count));

}
```

```
import urllib.request
import random
from operator import itemgetter

current_word = {}
current_count = 0
story = 'http://sixty-north.com/c/t.txt'
request = urllib.request.Request(story)
response = urllib.request.urlopen(request)
each_word = []
words = None
count = 1
same_words = {}
word = []
""" looping the entire file """
#Collect All the words into a list
for line in response:
    #print "Line = " , line
    line_words = line.split()
    for word in line_words: # looping each line and extracting words
        each_word.append(word)

#for every word collected, in dict same_words
#if a key exists, such that key == word then increment Mapping Value by 1
# Else add word as new key with mapped value as 1
for words in each_word:
    if words.lower() not in same_words.keys() :
        same_words[words.lower()]=1
    else:
        same_words[words.lower()]=same_words[words.lower()]+1

for each in same_words.keys():
    print ("word = ",each, ", count = ",same_words[each])
```

#Output :

```
word = b'incredulity' , count = 1
word = b'season' , count = 2
word = b'light' , count = 1
word = b'darkness' , count = 1
word = b'spring' , count = 1
word = b'hope' , count = 1
word = b'winter' , count = 1
word = b'despair' , count = 1
word = b'we' , count = 4
word = b'had' , count = 2
word = b'everything' , count = 1
word = b'before' , count = 2
word = b'us' , count = 2
word = b'nothing' , count = 1
word = b'were' , count = 2
word = b'all' , count = 2
word = b'going' , count = 2
word = b'direct' , count = 2
word = b'to' , count = 1
word = b'heaven' , count = 1
word = b'other' , count = 1
word = b'way' , count = 1
word = b'in' , count = 2
word = b'short' , count = 1
word = b'period' , count = 2
word = b'so' , count = 1
word = b'far' , count = 1
word = b'like' , count = 1
word = b'present' , count = 1
word = b'that' , count = 1
word = b'some' , count = 1
word = b'its' , count = 2
```

Practical-3

AIM :- TO Study Basic CRUD operation in MONGO DB.

DESCRIPTION :

MongoDB CRUD operations

C → **Create**

R → **Read**

U → **Update**

D → **Delete**

create database

use st

show dbs

Here, your created database "st" is not present in the list, insert at least one document into it to display database:

Insert

```
db.student.insert({"rollno":1,"Name":"raja"})
```

show dbs

st

Drop Database

```
db.dropDatabase()
```

MongoDB Create Collection

```
db.createCollection("SSSIT")  
db.SSSIT.insert({"name":"Raja"})  
db.SSSIT.find()
```

MongoDB Drop collection

```
show collections  
db.SSSIT.drop()
```

MongoDB insert documents

```
n1=[{"Name":"babu"}, {"Name":"karthi"}]  
db.student.insert(n1)  
db.student.find()
```

MongoDB update documents

```
db.student.update({'Name':'babu'},{$set: {'Name':'android'}})
```

MongoDB Delete documents

Remove all documents

```
db.student.remove({})
```

Remove all documents that match a condition

```
db.student.remove({"Name":"babu"})
```

Remove a single document that match a condition

```
db.student.remove({"Name":"babu"},1)
```

```
st> db.student.insert({"rollno":1,"Name":"OM BALAR"})
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('68c45a1b8ea6595bf5893bf9') }
}
st> db.dropDatabase()
{ ok: 1, dropped: 'st' }
st> db.createCollection("SSSIT")
{ ok: 1 }
st> db.SSSIT.insert({"name":"OM BALAR"})
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('68c45a3d8ea6595bf5893bfa') }
}
st> db.SSSIT.find()
[ { _id: ObjectId('68c45a3d8ea6595bf5893bfa'), name: 'OM BALAR' } ]
st> show collections
SSSIT
st> db.SSSIT.drop()
true
st> n1=[{"Name":"babu"}, {"Name":"karthi"}]
[ { Name: 'babu' }, { Name: 'karthi' } ]
st> db.student.insert(n1)
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('68c45a4e8ea6595bf5893bfb'),
    '1': ObjectId('68c45a4e8ea6595bf5893bfc')
  }
}
st> db.student.find()
[
  { _id: ObjectId('68c45a4e8ea6595bf5893bfb'), Name: 'babu' },
  { _id: ObjectId('68c45a4e8ea6595bf5893bfc'), Name: 'karthi' }
]
st> db.student.update({'Name':'babu'},{$set:{'Name':'android'}})
DeprecationWarning: Collection.update() is deprecated. Use updateOne, updateMany, or bulkWrite.
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
st> db.student.remove({})
DeprecationWarning: Collection.remove() is deprecated. Use deleteOne, deleteMany, findOneAndDelete, or bulkWrite.
{ acknowledged: true, deletedCount: 2 }
st> db.student.remove({"Name":"babu"})
{ acknowledged: true, deletedCount: 0 }
st> db.student.remove({"Name":"babu"},1)
{ acknowledged: true, deletedCount: 0 }
st> |
```

Comparison Query Operators

Name	Description
<u>\$eq</u>	Matches values that are equal to a specified value.
<u>\$gt</u>	Matches values that are greater than a specified value.
<u>\$gte</u>	Matches values that are greater than or equal to a specified value.
<u>\$in</u>	Matches any of the values specified in an array.
<u>\$lt</u>	Matches values that are less than a specified value.
<u>\$lte</u>	Matches values that are less than or equal to a specified value.
<u>\$ne</u>	Matches all values that are not equal to a specified value.
<u>\$nin</u>	Matches none of the values specified in an array.

Syntax

```
{field: { $gt: value } }
```

AND in MongoDB

```
db.mycol.find( { $and: [ {<key1>:<value1>}, { <key2>:<value2>} ] } )
```

OR in MongoDB

```
db.mycol.find( { $or: [ {<key1>:<value1>}, { <key2>:<value2>} ] } )
```

NOT in MongoDB

```
db.empDetails.find( { "Age": { $not: { $gt: "25" } } } )
```

The Limit() Method

```
db.COLLECTION_NAME.find().limit(NUMBER)
```

MongoDB Skip() Method

```
db.COLLECTION_NAME.find().limit(NUMBER).skip(NUMBER)
```

The sort() Method

To sort documents in MongoDB, you need to use sort() method. The method accepts a document containing a list of fields along with their sorting order. To specify sorting order 1 and -1 are used. 1 is used for ascending order while -1 is used for descending order.

```
db.COLLECTION_NAME.find().sort({KEY:1})
```

Lab Experiment

Designing an Amazon consumer review system using MongoDB as a storage engine. Amazon consumer review system must have the capacity to store many differed types of objects with different sets of attributes. These kinds of data collections are quite compatible with MongoDB's data model: Use a single MongoDB collection to store all the consumer reviews for Amazon products like the Kindle, TV and mobile. MongoDB's dynamic schema means that each document need not conform to the same schema.

1. Create Amazon Database using MongoDB and create a customer collection in the Amazon database.
2. Store the product review in customer collection.
3. Print the all customers review details
4. Find all the customer review of Samsung mobile product and print.
5. Print the all the customer review whose rating is more than 4.
6. Delete the customer whose name is Dhiraj.

```
switched to db Amazon
Amazon> db.createCollection("customer")
{ ok: 1 }
Amazon> db.customer.insertMany([
...   { name: "Ravi", product: "Kindle", brand: "Amazon", review: "Very useful for reading books.", rating: 5 },
...   { name: "Neha", product: "TV", brand: "LG", review: "Good picture quality but average sound.", rating: 4 },
...   { name: "Om Salun", product: "Mobile", brand: "Samsung", review: "Excellent battery life and camera.", rating: 5 },
...   { name: "Anit", product: "Mobile", brand: "Samsung", review: "Performance is smooth, but heating issue.", rating: 3 },
...   { name: "Pooja", product: "Mobile", brand: "Apple", review: "Very smooth experience, best in class.", rating: 5 }
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('68c45bb6dd1e6298b7893bf8'),
    '1': ObjectId('68c45bb6dd1e6298b7893bf9'),
    '2': ObjectId('68c45bb6dd1e6298b7893bfa'),
    '3': ObjectId('68c45bb6dd1e6298b7893bfb'),
    '4': ObjectId('68c45bb6dd1e6298b7893bfc')
  }
}
Amazon> db.customer.find().pretty()
[
  {
    _id: ObjectId('68c45bb6dd1e6298b7893bf8'),
    name: 'Ravi',
    product: 'Kindle',
    brand: 'Amazon',
    review: 'Very useful for reading books.',
    rating: 5
  },
  {
    _id: ObjectId('68c45bb6dd1e6298b7893bf9'),
    name: 'Neha',
    product: 'TV',
    brand: 'LG',
    review: 'Good picture quality but average sound.',
    rating: 4
  },
  {
    _id: ObjectId('68c45bb6dd1e6298b7893bfa'),
    name: 'Om Salun',
    product: 'Mobile',
    brand: 'Samsung',
    review: 'Excellent battery life and camera.',
    rating: 5
  },
  {
    _id: ObjectId('68c45bb6dd1e6298b7893bfb'),
    name: 'Anit',
    product: 'Mobile',
    brand: 'Samsung',
    review: 'Performance is smooth, but heating issue.',
    rating: 3
  },
  {
    _id: ObjectId('68c45bb6dd1e6298b7893bfc'),
    name: 'Pooja',
    product: 'Mobile',
    brand: 'Apple',
    review: 'Very smooth experience, best in class.',
    rating: 5
  }
]
Amazon> db.customer.find({ product: "Mobile", brand: "Samsung" })
[
  {
    _id: ObjectId('68c45bb6dd1e6298b7893bfa'),
    name: 'Om Salun',
    product: 'Mobile',
    brand: 'Samsung',
    review: 'Excellent battery life and camera.',
    rating: 5
  },
  {
    _id: ObjectId('68c45bb6dd1e6298b7893bfb'),
    name: 'Anit',
    product: 'Mobile',
    brand: 'Samsung',
    review: 'Performance is smooth, but heating issue.',
    rating: 3
  }
]
Amazon> db.customer.find({ rating: { $gt: 4 } })
[
  {
    _id: ObjectId('68c45bb6dd1e6298b7893bf8'),
    name: 'Ravi',
    product: 'Kindle',
    brand: 'Amazon',
    review: 'Very useful for reading books.',
    rating: 5
  },
  {
    _id: ObjectId('68c45bb6dd1e6298b7893bfa'),
    name: 'Om Salun',
    product: 'Mobile',
    brand: 'Samsung',
    review: 'Excellent battery life and camera.',
    rating: 5
  },
  {
    _id: ObjectId('68c45bb6dd1e6298b7893bfc'),
    name: 'Pooja',
    product: 'Mobile',
    brand: 'Apple',
    review: 'Very smooth experience, best in class.',
    rating: 5
  }
]
Amazon> db.customer.remove({ name: "Ravi" })
DeprecationWarning: Collection.remove() is deprecated. Use deleteOne, deleteMany, findOneAndDelete, or bulkWrite.
{ acknowledged: true, deletedCount: 1 }
Amazon> |
```

2. Build database for the parul university based on the following query and data using Mongo DB.

1. Create Parul Database using MongoDB and create a student collection in the parul Database.
2. Insert the student feedback in student collection.
3. Print the all-student details.
4. Find all the student feedback of BDA Lab and print.
5. Print the all the faculty name whose rating is more than 4.
6. Find the feedback of Dhiraj student and print.
7. Find the student feedback whose address is Parul University Hostel.
8. Sort all the student feedback based on the name and print.
9. Delete Dhiraj student in the student collection of Parul database.
10. Delete the feedback of karthi.

```
Parul> db.createCollection('student')
{ ok: 1 }
Parul> db.student.insertMany([
...   { student_name: "Om Salar", subject: "BDA Lab", faculty_name: "Prof. Shivangi", rating: 5, address: "Parul University Hostel" },
...   { student_name: "Meha", subject: "CNS Lab", faculty_name: "Prof. Sharma", rating: 4, address: "Vadodara" },
...   { student_name: "Dhiraj", subject: "BDA Lab", faculty_name: "Prof. Patel", rating: 5, address: "Parul University Hostel" },
...   { student_name: "Karthi", subject: "Java Lab", faculty_name: "Prof. Singh", rating: 3, address: "Ahmedabad" },
...   { student_name: "Pooja", subject: "BDA Lab", faculty_name: "Prof. Mehta", rating: 5, address: "Parul University Hostel" }
... ])
{ acknowledged: true,
  insertedIds: {
    0: ObjectId('68c45d9dd1e6208b7893b4d'),
    1: ObjectId('68c45d9dd1e6208b7893bfe'),
    2: ObjectId('68c45d9dd1e6208b7893bff'),
    3: ObjectId('68c45d9dd1e6208b7893c00'),
    4: ObjectId('68c45d9dd1e6208b7893c01')
  }
}
Parul> db.student.find().pretty()
{
  _id: ObjectId('68c45d9dd1e6208b7893b4d'),
  student_name: 'Om Salar',
  subject: 'BDA Lab',
  faculty_name: 'Prof. Shivangi',
  rating: 5,
  address: 'Parul University Hostel'
},
{
  _id: ObjectId('68c45d9dd1e6208b7893bfe'),
  student_name: 'Meha',
  subject: 'CNS Lab',
  faculty_name: 'Prof. Sharma',
  rating: 4,
  address: 'Vadodara'
},
{
  _id: ObjectId('68c45d9dd1e6208b7893bff'),
  student_name: 'Dhiraj',
  subject: 'BDA Lab',
  faculty_name: 'Prof. Patel',
  rating: 5,
  address: 'Parul University Hostel'
},
{
  _id: ObjectId('68c45d9dd1e6208b7893c00'),
  student_name: 'Karthi',
  subject: 'Java Lab',
  faculty_name: 'Prof. Singh',
  rating: 3,
  address: 'Ahmedabad'
},
{
  _id: ObjectId('68c45d9dd1e6208b7893c01'),
  student_name: 'Pooja',
  subject: 'BDA Lab',
  faculty_name: 'Prof. Mehta',
  rating: 5,
  address: 'Parul University Hostel'
}
Parul> db.student.find({ subject: "BDA Lab" })
{
  _id: ObjectId('68c45d9dd1e6208b7893b4d'),
  student_name: 'Om Salar',
  subject: 'BDA Lab',
  faculty_name: 'Prof. Shivangi',
  rating: 5,
  address: 'Parul University Hostel'
},
{
  _id: ObjectId('68c45d9dd1e6208b7893bff'),
  student_name: 'Dhiraj',
  subject: 'BDA Lab',
  faculty_name: 'Prof. Patel',
  rating: 5,
  address: 'Parul University Hostel'
},
{
  _id: ObjectId('68c45d9dd1e6208b7893c01'),
  student_name: 'Pooja',
  subject: 'BDA Lab',
  faculty_name: 'Prof. Mehta',
  rating: 5,
  address: 'Parul University Hostel'
}
```

```
Parul> db.student.find({ rating: { $gt: 4 } }, { faculty_name: 1, _id: 0 })
[
  { faculty_name: 'Prof. Shivangi' },
  { faculty_name: 'Prof. Patel' },
  { faculty_name: 'Prof. Mehta' }
]
Parul> db.student.find({ student_name: "Dhiraj" })
[
  {
    _id: ObjectId('68c45d9dd1e6208b7893bff'),
    student_name: 'Dhiraj',
    subject: 'BDA Lab',
    faculty_name: 'Prof. Patel',
    rating: 5,
    address: 'Parul University Hostel'
  }
]
Parul> db.student.find({ address: "Parul University Hostel" })
[
  {
    _id: ObjectId('68c45d9dd1e6208b7893b4d'),
    student_name: 'Om Salar',
    subject: 'BDA Lab',
    faculty_name: 'Prof. Shivangi',
    rating: 5,
    address: 'Parul University Hostel'
  },
  {
    _id: ObjectId('68c45d9dd1e6208b7893bff'),
    student_name: 'Dhiraj',
    subject: 'BDA Lab',
    faculty_name: 'Prof. Patel',
    rating: 5,
    address: 'Parul University Hostel'
  },
  {
    _id: ObjectId('68c45d9dd1e6208b7893c01'),
    student_name: 'Pooja',
    subject: 'BDA Lab',
    faculty_name: 'Prof. Mehta',
    rating: 5,
    address: 'Parul University Hostel'
  }
]
Parul> db.student.find().sort({ student_name: 1 })
[
  {
    _id: ObjectId('68c45d9dd1e6208b7893b4d'),
    student_name: 'Om Salar',
    subject: 'BDA Lab',
    faculty_name: 'Prof. Patel',
    rating: 5,
    address: 'Parul University Hostel'
  },
  {
    _id: ObjectId('68c45d9dd1e6208b7893c00'),
    student_name: 'Karthi',
    subject: 'Java Lab',
    faculty_name: 'Prof. Singh',
    rating: 3,
    address: 'Ahmedabad'
  },
  {
    _id: ObjectId('68c45d9dd1e6208b7893bfe'),
    student_name: 'Meha',
    subject: 'CNS Lab',
    faculty_name: 'Prof. Sharma',
    rating: 4,
    address: 'Vadodara'
  },
  {
    _id: ObjectId('68c45d9dd1e6208b7893b4d'),
    student_name: 'Om Salar',
    subject: 'BDA Lab',
    faculty_name: 'Prof. Shivangi',
    rating: 5,
    address: 'Parul University Hostel'
  },
  {
    _id: ObjectId('68c45d9dd1e6208b7893c01'),
    student_name: 'Pooja',
    subject: 'BDA Lab',
    faculty_name: 'Prof. Mehta',
    rating: 5,
    address: 'Parul University Hostel'
  }
]
Parul> db.student.remove({ student_name: "Dhiraj" })
{ acknowledged: true, deletedCount: 1 }
Parul> db.student.remove({ student_name: "Karthi" })
{ acknowledged: true, deletedCount: 1 }
```

Practical – 4

Aim :- Store the basic information about students such as roll no,name, date of birth and address of student using various collection types such as List, Set and Map.

Create database studentdb and Make Collection With name "student" and Follow Queries.

Insert Records Into student Collection using insertOne() method :

```
db.student.insertOne({
  no: 1,
  name: "ST",
  dob: "1995-09-26",
  e_mail: "radhika_sharma.123@gmail.com",
  phone: "9848022338",
  address: { building: "1007", street: "vashna bhaily", zipcode: "10462" },
  Branch: "CSE",
  marks: [50, 90, 30]
})
```

Insert multiple documents :

```
db.student.insertMany([
  {
    no: 2,
    name: "ST",
    dob: "1995-09-26",
    e_mail: "radhika_sharma.123@gmail.com",
    phone: "9848022338",
    address: { building: "1008", street: "vashna bhaily", zipcode: "10462" },
    Branch: "CSE",
    marks: [50, 90, 30]
  },
  {
    no: 3,
    name: "karthi",
    dob: "1995-09-26",
    e_mail: "radhika_sharma.123@gmail.com",
    phone: "9448022338",
    address: { building: "1009", street: "vashna bhaily", zipcode: "10462" },
    Branch: "CSE",
    marks: [50, 90, 12]
  }
])
```

List all documents :

```
db.student.find()
```

Update document

Update student name where no = 1:

```
db.student.update(
  { no: 1 },
  { $set: { name: "sita" } }
)
```

Querying with Regex

Name begins with **S** :

```
db.student.find({ name: /^S/ })
```

Name ends with **I** :

```
db.student.find({ name: /i$/ })
```

Name contains **S** anywhere :

```
db.student.find({ name: /S/ })
```

Count documents

```
db.student.count()
```

Using forEach()

Print all names :

```
db.student.find().forEach(function(myDoc) {  
  print("name: " + myDoc.name);  
});
```

Limit results

```
db.student.find().limit(2)
```

Sort results

Sort by name in descending order :

```
db.student.find().sort({ name: -1 })
```

\$map :

Applies an expression to each item in an array and returns an array with the applied results.

```
{ $map: { input: <expression>, as: <string>, in: <expression> } }
```

\$map Example :-

Insert into grades :

```
db.grades.insertMany([  
  { _id: 1, quizzes: [5, 6, 7] },  
  { _id: 2, quizzes: [] },  
  { _id: 3, quizzes: [3, 8, 9] }  
])
```

Aggregate with \$map (adds +2 to each quiz) :

```
db.grades.aggregate([
  {
    $project: {
      adjustedGrades: {
        $map: {
          input: "$quizzes",
          as: "grade",
          in: { $add: ["$$grade", 2] }
        }
      }
    }
  }
])
```

Output:

```
{ "_id": 1, "adjustedGrades": [7, 8, 9] }
{ "_id": 2, "adjustedGrades": [] }
{ "_id": 3, "adjustedGrades": [5, 10, 11] }
```

set

Adds new fields to documents. \$set outputs documents that contain all existing fields from the input documents and newly added fields.

\$set Example :

Insert into scores:

```
db.scores.insertMany([
  { _id: 1, student: "Maya", homework: [10, 5, 10], quiz: [10, 8], extraCredit: 0 },
  { _id: 2, student: "Ryan", homework: [5, 6, 5], quiz: [8, 8], extraCredit: 8 }
])
```

Aggregate with \$set:

```
db.scores.aggregate([
  {
    $set: {
      totalHomework: { $sum: "$homework" },
      totalQuiz: { $sum: "$quiz" }
    }
  },
  {
    $set: {
      totalScore: { $add: ["$totalHomework", "$totalQuiz", "$extraCredit"] }
    }
  }
])
```

Lab Experiment

Designing an employee management system using MongoDB as a storage engine. employee management system must have the capacity to store many differed types of objects with different sets of attributes. These kinds of data collections are quite compatible with MongoDB's data model: Use a single MongoDB collection to store all the student feedback for the parul university like the employee name, ID and salary. MongoDB's dynamic schema means that each document need not conform to the same schema.

1. Create parul Database using MongoDB and create a employee collection in the parul Database.

Create Database

```
test> use Parul;
switched to db Parul
```

Create Collection & Insert Employee Records

```
db.employee.insertMany([
{
  emp_id: 101,name: "Om",basic: 30000,hra: 5000,da: 4000,income_tax: 2000,prof_tax: 500
},
{
  emp_id: 102,name: "Neha",basic: 40000,hra: 6000,da: 5000,income_tax: 3000,prof_tax: 800
},
{
  emp_id: 103,name: "Amit",basic: 25000,hra: 4000,da: 3000,income_tax: 1500, prof_tax: 400
}
])
```

```
Parul> db.employee.insertMany([
...   {
...     emp_id: 101,
...     name: "Om",
...     basic: 30000,
...     hra: 5000,
...     da: 4000,
...     income_tax: 2000,
...     prof_tax: 500
...   },
...   {
...     emp_id: 102,
...     name: "Neha",
...     basic: 40000,
...     hra: 6000,
...     da: 5000,
...     income_tax: 3000,
...     prof_tax: 800
...   },
...   {
...     emp_id: 103,
...     name: "Amit",
...     basic: 25000,
...     hra: 4000,
...     da: 3000,
...     income_tax: 1500,
...     prof_tax: 400
...   }
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('68c4537f4803c1f800893bf8'),
    '1': ObjectId('68c4537f4803c1f800893bf9'),
    '2': ObjectId('68c4537f4803c1f800893bfa')
  }
}
```

2. Write the basic salary ,HRA, DA, Income Tax , Professional Tax of an employee and calculate its Gross salary of employee using aggregate functions (Gross salary=Basic Salary + HRA + DA - Income Tax –Professional Tax).

```
db.employee.aggregate([
{
  $set: {
    gross_salary: {
      $subtract: [
        { $add: ["$basic", "$hra", "$da"] },
        { $add: ["$income_tax", "$prof_tax"] }
      ]
    }
  }
}]
)
```

```
Parul> db.employee.aggregate([
... {
...   $set: {
...     gross_salary: {
...       $subtract: [
...         { $add: ["$basic", "$hra", "$da"] },
...         { $add: ["$income_tax", "$prof_tax"] }
...       ]
...     }
...   }
... }
... ])
[
{
```

Expected Output :

```
{
  _id: ObjectId('68c4537f4803c1f800893bf8'),
  emp_id: 101,
  name: 'Om',
  basic: 30000,
  hra: 5000,
  da: 4000,
  income_tax: 2000,
  prof_tax: 500,
  gross_salary: 36500
},
{
  _id: ObjectId('68c4537f4803c1f800893bf9'),
  emp_id: 102,
  name: 'Neha',
  basic: 40000,
  hra: 6000,
  da: 5000,
  income_tax: 3000,
  prof_tax: 800,
  gross_salary: 47200
},
{
  _id: ObjectId('68c4537f4803c1f800893bfa'),
  emp_id: 103,
  name: 'Amit',
  basic: 25000,
  hra: 4000,
  da: 3000,
  income_tax: 1500,
  prof_tax: 400,
  gross_salary: 30100
}
]
Parul> |
```

Practical – 5

AIM :- To study Basic commands available for the Hadoop Distributed File System.

HDFS Commands

HDFS is the primary or major component of the Hadoop ecosystem which is responsible for storing large data sets of structured or unstructured data across various nodes and thereby maintaining the metadata in the form of log files. To use the HDFS commands, first you need to start the Hadoop services using the following command:

start-all.sh
stop-all.sh

hadoop version :

The Hadoop fs shell command version prints the Hadoop version.

Jps :

To check the Hadoop services are up and running use the following command:

```
(base) dhanra_j100@MacBook-Air ~ % jps
26055 Jns
```

ls: This command is used to list all the files.

```
(base) dhanra_j100@MacBook-Air ~ % ls
abc.html
AndroidStudioProjects
Applications
ASCII
AVL_GUI.py
BDA_1.ipynb
CI.py
Desktop
Diwali Sales Data.csv
Documents
Downloads
eclipse
eclipse-workspace
```

hadoop fs -ls

It will print all the directories present in HDFS. bin directory contains executables so,

```
(base) dhanra_j100@MacBook-Air ~ % hadoop fs -ls
2025-09-12 22:02:14,945 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform...
using builtin-java classes where applicable
Found 107 items
-r----- 1 dhanra_j100 staff 7 2023-02-16 17:42 .CFUserTextEncoding
-rw-r--r-- 1 dhanra_j100 staff 18436 2025-09-12 21:49 .DS_Store
drwx----- - dhanra_j100 staff 1920 2025-09-10 12:05 .Trash
drwxr-xr-x - dhanra_j100 staff 160 2024-12-05 09:54 .anaconda
drwxr-xr-x - dhanra_j100 staff 416 2025-01-17 13:49 .android
drwxr-xr-x - dhanra_j100 staff 320 2025-03-23 12:55 .arduinoIDE
-rw-r--r-- 1 root staff 447 2024-07-03 08:10 .bash_profile
drwxr-xr-x - dhanra_j100 staff 160 2025-03-27 22:09 .cache
drwxr-xr-x - dhanra_j100 staff 128 2025-08-18 14:41 .conda
-rw-r--r-- 1 dhanra_j100 staff 23 2024-07-03 08:11 .condarc
```

mkdir:

To create a directory. In Hadoop dfs there is no home directory by default. So let's first create it.

hadoop dfs -mkdir bdalab

vi lab.txt

cat lab.txt

creating local file and viewing the content.

```
(base) dhanra_j100@MacBook-Air ~ % hadoop fs -mkdir /Users/dhanra_j100/bdalab

2025-09-12 23:03:29,033 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
(base) dhanra_j100@MacBook-Air ~ % hadoop fs -ls /Users/dhanra_j100

2025-09-12 23:04:27,893 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 107 items
-r----- 1 dhanra_j100 staff      7 2023-02-16 17:42 /Users/dhanra_j100/.CFUserTextEncoding
-rw-r--r-- 1 dhanra_j100 staff 18436 2025-09-12 21:49 /Users/dhanra_j100/.DS_Store
drwx-----  - dhanra_j100 staff 1920 2025-09-10 12:05 /Users/dhanra_j100/.Trash
drwxr-xr-x  - dhanra_j100 staff 160 2024-12-05 09:54 /Users/dhanra_j100/.anaconda
drwxr-xr-x  - dhanra_j100 staff 416 2025-01-17 13:49 /Users/dhanra_j100/.android
drwxr-xr-x  - dhanra_j100 staff 256 2024-07-20 13:12 /Users/dhanra_j100/.VS Code
drwxr-xr-x  - dhanra_j100 staff 96 2025-07-02 16:11 /Users/dhanra_j100/VirtualBox VMs
-rw-r--r-- 1 dhanra_j100 staff 21037 2025-06-28 08:39 /Users/dhanra_j100/abc.html
drwxr-xr-x  - dhanra_j100 staff 64 2025-09-12 23:03 /Users/dhanra_j100/bdalab
drwxr-xr-x  - dhanra_j100 staff 128 2024-07-03 10:57 /Users/dhanra_j100/eclipse
drwxr-xr-x  - dhanra_j100 staff 352 2024-10-23 08:32 /Users/dhanra_j100/eclipse-workspace
```

put

To copy files/folders from local file system to hdfs store. This is the most important command. Local filesystem means the files present on the OS.

syntax

`hadoop fs -put <localsrc> <dest>;`

```
(base) dhanra_j100@MacBook-Air ~ % echo "Hello Hadoop Practical" > lab.txt
hadoop fs -put lab.txt /Users/dhanra_j100/bdalab

2025-09-12 23:10:56,355 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
(base) dhanra_j100@MacBook-Air ~ % hadoop fs -ls /Users/dhanra_j100/bdalab

2025-09-12 23:11:06,058 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 1 items
-rw-r--r-- 1 dhanra_j100 staff      23 2025-09-12 23:10 /Users/dhanra_j100/bdalab/lab.txt
```

<http://localhost:50070/>

to check the file copied to Hadoop file system or not in the graphical user interface.

copyToLocal (or) get: To copy files/folders from hdfs store to local file system.

Syntax:

`Hadoop fs -get <srcfile(on hdfs)> <local file dest>;`

```
(base) dhanra_j100@MacBook-Air ~ % hadoop fs -get /Users/dhanra_j100/bdalab/local_test.txt local_copy.txt
cat local_copy.txt

2025-09-12 23:18:54,391 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
This is Hadoop practical
```

moveFromLocal: This command will move file from local to hdfs.

Syntax:

`Hadoop fs -moveFromLocal <local src> <dest(on hdfs)>;`

```
(base) dhanra_j100@MacBook-Air ~ % echo "This is Hadoop practical" > local_test.txt
hadoop fs -moveFromLocal local_test.txt /Users/dhanra_j100/bdalab/

2025-09-12 23:17:44,301 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
```

Example:

`hadoop fs -moveFromLocal /home/user/Desktop/test/t.txt /karthi`

cp: This command is used to copy files within hdfs. Lets copy folder geeks to geeks_copied.

Syntax:

`Hadoop -fs -cp <src(on hdfs)> <dest(on hdfs)>;`

mv: This command is used to move files within hdfs.

Syntax:

`Hadoop fs -mv <src(on hdfs)> <src(on hdfs)>;`

rm: This command deletes a file from HDFS.

Syntax:

Hadoop fs -rm <filename/directoryName>;

Hadoop fs -rmr /directory ->; It will delete all the content inside the directory then the directory itself.

du: It will give the size of each file in directory.

Syntax:

Hadoop fs -du <dirName>;

Example:

dfs: This command will give the total size of directory/file.

Syntax:

Hadoop fs -dfs <dirName>;

```
(base) dhanra_j100@MacBook-Air ~ % cp lab.txt bdalab/
(base) dhanra_j100@MacBook-Air ~ % mv lab.txt bdalab/lab_moved.txt
(base) dhanra_j100@MacBook-Air ~ % cp bdalab/lab_moved.txt bdalab/lab_copy.txt
(base) dhanra_j100@MacBook-Air ~ % rm bdalab/lab_copy.txt
(base) dhanra_j100@MacBook-Air ~ % du -sh bdalab
8.0K  bdalab
```

```
(base) dhanra_j100@MacBook-Air ~ % hadoop fs -dfs /Users/dhanra_j100/bdalab
2025-09-12 23:13:46,438 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
dfs: DEPRECATED: Please use 'du -s' instead.
23 23 /Users/dhanra_j100/bdalab
```

stat: It will give the last modified time of directory or path. In short it will give stats of the directory or file.

Syntax:

Hadoop fs -stat <hdfs file>;

```
(base) dhanra_j100@MacBook-Air ~ % hadoop fs -stat /Users/dhanra_j100/bdalab/local_test.txt
2025-09-12 23:30:43,577 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2025-09-12 17:47:44
```

setrep: This command is used to change the replication factor of a file/directory in HDFS. By default, it is 3 for anything which is stored in HDFS (as set in hdfs core-site.xml).

Example 1: To change the replication factor to 6 for geeks.txt stored in HDFS.

Hadoop fs -setrep -R -w 6 test

```
(base) dhanra_j100@MacBook-Air ~ % hadoop fs -setrep -R -w 2 /Users/dhanra_j100/bdalab/local_test.txt
2025-09-12 23:24:07,837 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Replication 2 set: /Users/dhanra_j100/bdalab/local_test.txt
Waiting for /Users/dhanra_j100/bdalab/local_test.txt .....
```

Note: -R means recursively, we use it for directories as they may also contain many files and folders inside them.

test

The test command is used for file test operations.

Options Description

- d Check whether the path given by the user is a directory or not, return 0 if it is a directory.
- e Check whether the path given by the user exists or not, return 0 if the path exists.
- f Check whether the path given by the user is a file or not, return 0 if it is a file.
- s Check if the path is not empty, return 0 if a path is not empty.
- r return 0 if the path exists and read permission is granted
- w return 0 if the path exists and write permission is granted
- z Checks whether the file size is 0 byte or not, return 0 if the file is of 0 bytes.

```
(base) dhanra_j100@MacBook-Air ~ % hadoop fs -test -d /Users/dhanra_j100/bdalab && echo "It is a directory"
2025-09-12 23:22:41,143 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
It is a directory
(base) dhanra_j100@MacBook-Air ~ % hadoop fs -test -f /Users/dhanra_j100/bdalab/local_test.txt && echo "It is a file"
2025-09-12 23:22:50,425 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
It is a file
```

Getmerge:

getmerge command merges a list of files in a directory on the HDFS filesystem into a single local file on the local filesystem.

```
(base) dhanra_j100@MacBook-Air ~ % echo "line1" > file1.txt
echo "line2" > file2.txt
hadoop fs -put file1.txt /Users/dhanra_j100/bdalab
hadoop fs -put file2.txt /Users/dhanra_j100/bdalab
hadoop fs -getmerge /Users/dhanra_j100/bdalab merged.txt
cat merged.txt
2025-09-12 23:27:57,562 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2025-09-12 23:27:58,085 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2025-09-12 23:27:58,600 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
line1
line2
Hello Hadoop Practical
This is Hadoop practical
```

stat prints the statistics about the file or directory in the specified format.

Formats:

- %b – file size in bytes
- %g – group name of owner
- %n – file name
- %o – block size
- %r – replication
- %u – user name of owner
- %y – modification date

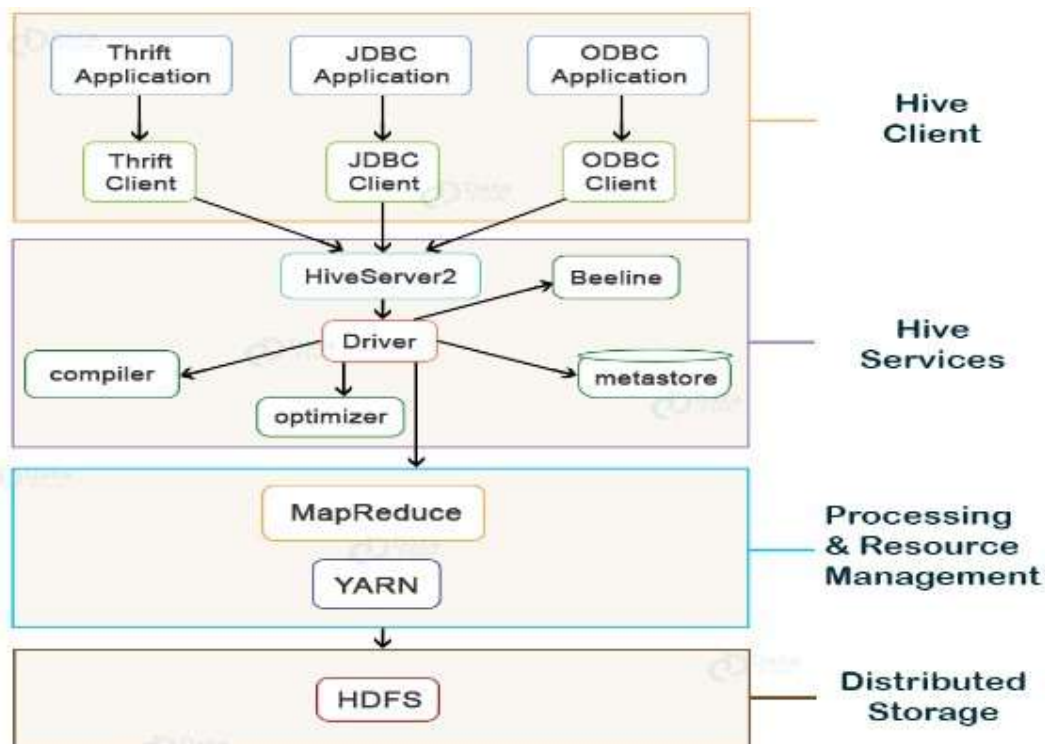
```
(base) dhanra_j100@MacBook-Air ~ % hadoop fs -stat /Users/dhanra_j100/bdalab/local_test.txt
2025-09-12 23:20:13,759 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2025-09-12 17:47:44
(base) dhanra_j100@MacBook-Air ~ % hadoop fs -stat %g /Users/dhanra_j100/bdalab/local_test.txt
2025-09-12 23:20:30,946 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
staff
(base) dhanra_j100@MacBook-Air ~ % hadoop fs -stat %n /Users/dhanra_j100/bdalab/local_test.txt
2025-09-12 23:20:41,612 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
local_test.txt
(base) dhanra_j100@MacBook-Air ~ % hadoop fs -stat %b /Users/dhanra_j100/bdalab/local_test.txt
2025-09-12 23:20:50,175 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
25
(base) dhanra_j100@MacBook-Air ~ % hadoop fs -stat %o /Users/dhanra_j100/bdalab/local_test.txt
2025-09-12 23:20:59,147 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
33554432
```

Practicle-6

Aim :- To study basic commands available for HIVE Query Language.

Description:

Apache Hive is an open-source data warehousing tool for performing distributed processing and data analysis. It was developed by Facebook to reduce the work of writing the Java MapReduce program. Apache Hive uses a Hive Query language, which is a declarative language similar to SQL. Hive translates the hive queries into MapReduce programs. It supports developers to perform processing and analyses on structured and semi-structured data by replacing complex java MapReduce programs with hive queries. One who is familiar with SQL commands can easily write the hive queries.



Hive Architecture & Its Components

Hive supports applications written in any language like Python, Java, C++, Ruby, etc. using JDBC, ODBC, and Thrift drivers, for performing queries on the Hive. Hence, one can easily write a hive client application in any language of its own choice.

Hive clients are categorized into three types:

1. The Hive server is based on Apache Thrift so that it can serve the request from a thrift client.
2. JDBC client

Hive allows for the Java applications to connect to it using the JDBC driver. JDBC driver uses Thrift to communicate with the Hive Server.

3. ODBC client

Hive ODBC driver allows applications based on the ODBC protocol to connect to Hive. Similar to the JDBC driver, the ODBC driver uses Thrift to communicate with the Hive Server.

Hive - Create Database

In Hive, the database is considered as a catalog or namespace of tables. So, we can maintain multiple tables within a database where a unique name is assigned to each table. Hive also provides a default database with a name default.

Initially, we check the default database provided by Hive. So, to check the list of existing databases, follow the below command: -

```
hive> show databases;
```

```
hive> create database demo;
```

```
hive> show databases;
```

```
hive> describe database extended demo;
```

Hive - Create Table

In Hive, we can create a table by using the conventions similar to the SQL. It supports a wide range of flexibility where the data files for tables are stored. It provides two types of table: -

Internal table

The internal tables are also called managed tables as the lifecycle of their data is controlled by the Hive. By default, these tables are stored in a subdirectory under the directory defined by hive.metastore.warehouse.dir (i.e. /user/hive/warehouse). The internal tables are not flexible enough to share with other tools like Pig. If we try to drop the internal table, Hive deletes both table schema and data

```
hive> create table demo.employee (Id int, Name string , Salary float)
```

```
row format delimited
```

```
fields terminated by ',' ;
```

External Table

The external table allows us to create and access a table and a data externally. The external keyword is used to specify the external table, whereas the location keyword is used to determine the location of loaded data. As the table is external, the data is not present in the

Hive directory. Therefore, if we try to drop the table, the metadata of the table will be deleted, but the data still exists.

Let's create a directory on HDFS by using the following command: -

```
hadoop dfs -mkdir /HiveDirectory
```

Now, store the file on the created directory.

```
Hadoop dfs -put hive/emp_details /HiveDirectory
```

```
Hive> create external table emplist (Id int, Name string , Salary float)
```

```
row format delimited
```

```
fields terminated by ',';
```

```
location '/HiveDirectory';
```

```
select * from emplist;
```

```
hive> select * from emplist;  
OK  
1      "Gaurav"      30000.0  
2      "Aryan" 20000.0  
3      "Vishal"      40000.0  
Time taken: 7.096 seconds, Fetched: 3 row(s)  
hive>
```

Hive - Load Data

Once the internal table has been created, the next step is to load the data into it. So, in Hive, we can easily load data from any file to the database.

```
load data local inpath '/home/codegyani/hive/emp_details' into table demo.employee;
```

```
select * from demo.employee;
```

Hive - Drop Table

Hive facilitates us to drop a table by using the SQL drop table command. Let's follow the below steps to drop the table from the database.

```
show databases;
```

```
use demo;
```

Practical – 7

AIM :- Basic commands of HBASE Shell

Description

HBase is a **distributed, column-oriented database** built on top of the Hadoop File System (HDFS). It is an open-source project designed for **horizontal scalability**. HBase provides a data model similar to Google's BigTable and allows **real-time read/write access** to huge amounts of structured data.

It leverages the **fault tolerance** of HDFS and forms an important part of the Hadoop ecosystem. While data can be stored in HDFS directly, HBase provides a mechanism to **store and access data randomly** with high efficiency. Essentially, HBase **sits on top of HDFS** and offers read and write operations on large datasets.

Data Definition Language (DDL)

Create

```
create 'emp', 'personal data', 'professional data'
```

List

```
List
```

Disable

```
disable 'emp'
```

Check if Disabled

```
is_disabled 'emp'
```

Enable

```
enable 'emp'
```

Check if Enabled

```
is_enabled 'emp'
```

Describe

```
describe 'emp'
```

Drop

```
drop 'emp'
```

Data Manipulation Language (DML)

Put

```
put 'emp','1','personal data:name','raju'  
put 'emp','1','personal data:city','hyderabad'  
put 'emp','1','professional data:designation','manager'  
put 'emp','1','professional data:salary','50000'  
put 'emp','1','professional data:vechiv','50000'  
put 'emp','2','personal data:name','sathish'  
put 'emp','2','personal data:city','bangalore'  
put 'emp','2','professional data:designation','professor'  
put 'emp','2','professional data:salary','60000'
```

```
put 'emp','3','personal data:name','muthu'  
put 'emp','3','personal data:city','chennai'  
put 'emp','3','professional data:designation','analyst'  
put 'emp','3','professional data:salary','20000'
```

Get

```
get 'emp','1'
```

Delete

```
delete 'emp','1','personal data:city',1417521848375
```

Delete All

```
deleteall 'emp','1'
```

Scan

```
scan 'emp'
```

Count

```
count 'emp'
```

Truncate

```
truncate 'emp'
```

Practical – 8

AIM :- Creating the HDFS tables and loading them in Hive and learn join, partition of tables in Hive.

Description:

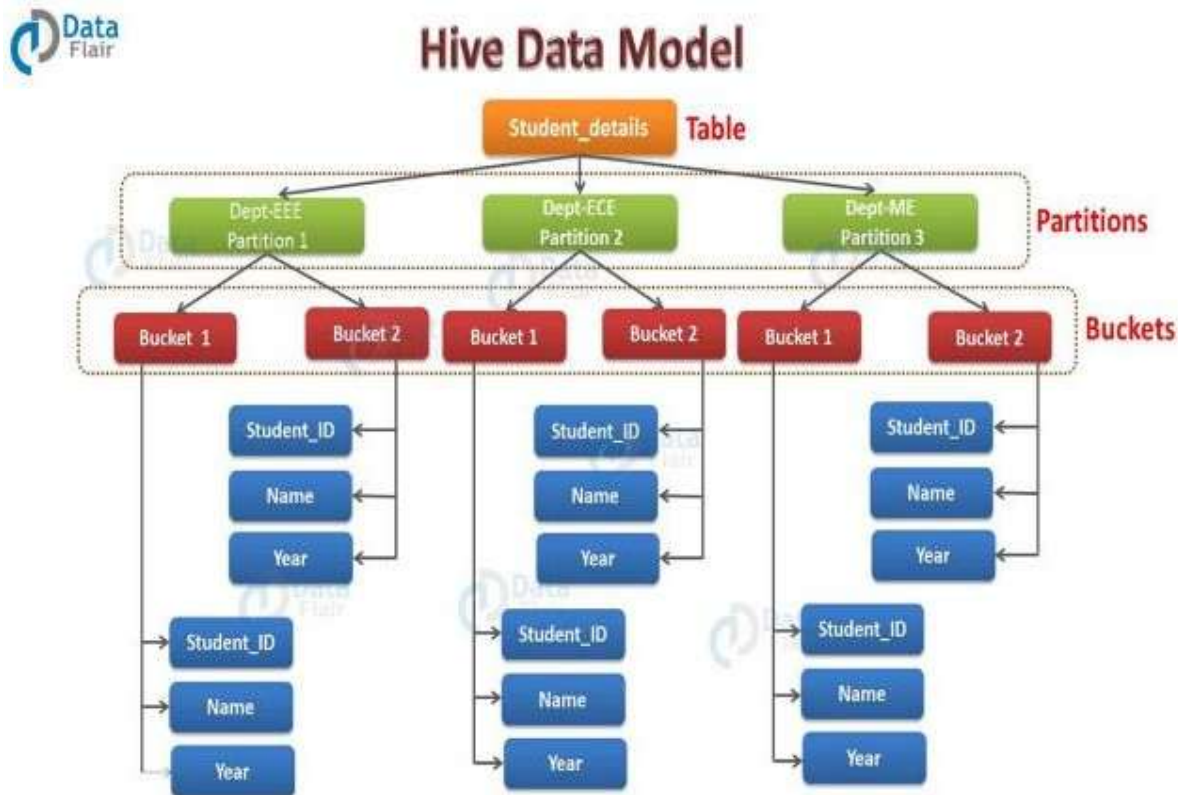
Partitions

Each table can be broken into partitions. Partitions determine distribution of data within subdirectories. In the current century, we know that the huge amount of data which is in the range of petabytes is getting stored in HDFS. Due to this, it becomes very difficult for Hadoop users to query this huge amount of data.

Hive was introduced to lower down this burden of data querying. Apache Hive converts SQL queries into MapReduce jobs and then submits them to the Hadoop cluster. When we submit a SQL query, Hive reads the entire dataset. So, it becomes inefficient to run MapReduce jobs over a large table. This is resolved by creating partitions in tables.

Apache Hive makes partitioning very easy by creating partitions using its automatic partition scheme at the time of table creation.

In partitioning method, all the table data is divided into multiple partitions. Each partition corresponds to a specific value(s) of partition column(s). It is kept as a sub-record inside the table's record present in the HDFS. Therefore, on querying a particular table, only the appropriate partition of the table is queried which contains the query value. This decreases the I/O time required by the query and increases the performance speed.



Static Partitions

Static partition means inserting input data files individually into a partition table. Usually, when loading big files into Hive tables, static partitions are preferred. Static partition saves time in loading data compared to dynamic partition. You add a partition in the table statically and move the file into the partition of the table. We can alter the partition in static partitioning. The partition column value can be derived from the filename, day of date, etc., without reading the whole big file.

To use static partitions in Hive, set the property:
`set hive.mapred.mode = strict;`

This property is set by default in hive-site.xml. Static partition works in **strict mode**. You must use a WHERE clause to use limit in static partition. Static partitions can be performed on both managed and external Hive tables.

Dynamic Partitions

Dynamic partition means a single insert to a partitioned table. Usually, dynamic partitions load data from a non-partitioned table. Dynamic partition takes more time in loading data compared to static partition, but it is suitable for very large datasets. Dynamic partitioning is also useful when you want to partition a number of columns but you don't know how many columns beforehand. Unlike static partitioning, dynamic partitioning does not require a WHERE clause to use limit. However, we cannot perform ALTER on dynamic partitions. Dynamic partitioning can be used on both managed and external Hive tables. To enable it, we must set the mode to **non-strict mode**.

Commands to enable dynamic partitioning:

```
set hive.exec.dynamic.partition=true;  
set hive.exec.dynamic.partition.mode=nonstrict;
```

Hive Commands

1. Database Commands

```
create database test;  
use test;  
drop database test;  
show tables;  
drop table student;  
show databases;
```

2. Create Partitioned Table

```
create table student(name string, rollno int, percentage float)  
partitioned by(state string, city string)  
row format delimited  
fields terminated by ',';
```

3. Load Data into Static Partitions

```
load data local inpath '/home/training/Desktop/maharashtra'  
into table student partition(state='maharashtra', city='mumbai');
```

```
load data local inpath '/home/training/Desktop/karnataka'  
into table student partition(state='karnataka', city='bangalore');
```

4. Select Queries

```
select * from student;  
select * from student where state='maharastra';
```

Dynamic Partitioning Example

1. Enable dynamic partitioning

```
set hive.exec.dynamic.partition=true;  
set hive.exec.dynamic.partition.mode=nonstrict;
```

2. Create a source table

```
create table stu(name string, rollno int, percentage float, state string, city string)  
row format delimited  
fields terminated by ',';
```

3. Load data

```
load data local inpath '/home/training/Desktop/Result1' into table stu;
```

4. Create partitioned table

```
create table stud_part (name string, rollno int, percentage float)  
partitioned by (state string, city string)  
row format delimited  
fields terminated by ',';
```

5. Insert data dynamically

```
insert overwrite table stud_part  
partition(state, city)  
select name, rollno, percentage, state, city from stu;
```

6. Query

```
select * from stud_part where city='bangalore';
```

Sample Data**Karnataka.txt**

```
Rajesh,100,78  
Abhishek,95,76  
Manish,102,89  
Siva,203,66  
Sania,204,77
```

Maharastra.txt

```
Ravi,100,56  
Mohan,95,89  
Mahesh,102,67  
Janvi,103,66
```

Hive Joins

Let's see two tables Employee and Employee Department that are going to be joined.

Employee			Employee Department	
EMP ID	Emp Name	Address	Emp ID	Department
1	Rose	US	1	IT
2	Fred	US	2	IT
3	Jess	In	3	Eng
4	Frey	Th	4	Admin

Inner Join

```
select * from employee
join employeedepartment
on (employee.empid = employeedepartment.empid);
```

Hive Join

Let's see two tables Employee and EmployeeDepartment that are going to be joined.

Employee department table hive DML operation

Left Outer Join

```
select e.empid, empname, department
from employee e
left outer join employeedepartment ed
on (e.empid = ed.empid);
```

Right Outer Join

```
select e.empid, empname, department
from employee e
right outer join employeedepartment ed
on (e.empid = ed.empid);
```